

Ejercicios prácticos sobre funciones básicas (.map, .filter, .reduce)

A continuación se encuentran los ejercicios trabajados en clase de Programación III (tipado/coerción, mutabilidad/scope, **map**, **forEach**, **spread**, **filter** y **reduce**). Se trata de **4 ejercicios prácticos** incluida su solución. Al final se retoma el ejercicio 1 pero modificando el origen de datos por un archivo formato JSON.

Ejercicio 1: Procesador de Calificaciones y Reporte HTML (ejercicio01.js)

- **Objetivo:** Practicar transformación de objetos con `.map()`, operador `spread (...)`, creación de marcado HTML y evitar el uso incorrecto de `.map()`.
- **Consigna:** Dado el siguiente arreglo de alumnos:

JavaScript

```
const estudiantes = [  
  { id: 1, nombre: 'Lucas', parcial1: 4, parcial2: 6 },  
  { id: 2, nombre: 'Mariana', parcial1: 8, parcial2: 9 },  
  { id: 3, nombre: 'Gonzalo', parcial1: 2, parcial2: 3 },  
  { id: 4, nombre: 'Flores', parcial1: 7, parcial2: 5 }  
];
```

1. Utilizando **.map()** y el **operador Spread (...)**, generará un nuevo arreglo `estudiantesConPromedio` donde cada objeto conserve sus propiedades originales e incorpore:
 - `promedio`: Promedio numérico entre `parcial1` y `parcial2`.
 - `condicion`: String que valga `'APROBADO'` si el promedio es ≥ 6 o `'RECUPERA'` si es menor.
2. A partir del arreglo generado, utilizá **.map()** y **.join()** para crear una cadena de texto que genere una lista HTML con el formato: `<li>Lucas - Promedio: 5 (RECUPERA)</li>`
3. Imprimí el HTML resultante por consola y verificá que el arreglo `estudiantes` original no haya sufrido ninguna mutación.

Solución: ejercicio01.js

```
const estudiantes = [  
  { id: 1, nombre: 'Lucas', parcial1: 4, parcial2: 6 },  
  { id: 2, nombre: 'Mariana', parcial1: 8, parcial2: 9 },  
  { id: 3, nombre: 'Gonzalo', parcial1: 2, parcial2: 3 },  
  { id: 4, nombre: 'Flores', parcial1: 7, parcial2: 5 }  
];  
  
// 1. Transformación con .map() y operador Spread (...)  
const estudiantesConPromedio = estudiantes.map(estudiante => {  
  const promedio = (estudiante.parcial1 + estudiante.parcial2) / 2;  
  return {  
    ...estudiante,  
    promedio: promedio,  
    condicion: promedio >= 6 ? 'APROBADO' : 'RECUPERA'  
  };  
});
```

```
});

console.log('=== ESTUDIANTES CON PROMEDIO ===');
console.table(estudiantesConPromedio);

// 2. Generación del marcado HTML con .map() y .join()
const itemsHTML = estudiantesConPromedio.map(
  e => `<li>${e.nombre} - Promedio: ${e.promedio} (${e.condicion})</li>`
);

const reporteHTML = `<ul class="lista-estudiantes">\n${itemsHTML.join("\n")}\n</ul>`;

console.log("\n=== REPORTE HTML GENERADO ===");
console.log(reporteHTML);

// 3. Verificación de inmutabilidad
console.log("\n¿Arreglo original intacto?:", estudiantes[0].promedio === undefined);
```

Ejercicio 2: Auditoría y Filtrado de Inventario (ejercicio02.js)

- **Objetivo:** Dominar `.filter()`, la combinación de condiciones lógicas y entender la diferencia entre iterar para transformar vs. iterar para ejecutar acciones (`.forEach()`).
- **Consigna:** Dado el catálogo de productos de un depósito:

```
const inventario = [
  { id: 'P1', nombre: 'Sensor Inductivo', precio: 45, categoria: 'Electrónica', stock: 12 },
  { id: 'P2', nombre: 'Pulsador Industrial', precio: 12, categoria: 'Electricidad', stock: 0 },
  { id: 'P3', nombre: 'PLC Compacto', precio: 320, categoria: 'Electrónica', stock: 4 },
  { id: 'P4', nombre: 'Contactor 220V', precio: 85, categoria: 'Electricidad', stock: 15 },
  { id: 'P5', nombre: 'Relé Térmico', precio: 55, categoria: 'Electricidad', stock: 0 }
];
```

1. Utilizando `.filter()`, obtené un nuevo arreglo críticos con aquellos productos que pertenecen a la categoría 'Electricidad' Y tienen un stock igual a 0. Mostralo en consola con `console.table()`.
2. Utilizando `.forEach()`, recorré el arreglo de productos críticos e imprimí un mensaje de alerta por consola para el equipo de compras con el siguiente formato por cada ítem: [ALERTA REPOSICIÓN] El producto 'Pulsador Industrial' no tiene stock disponible.

Solución: ejercicio02.js

```
const inventario = [
  { id: 'P1', nombre: 'Sensor Inductivo', precio: 45, categoria: 'Electrónica', stock: 12 },
  { id: 'P2', nombre: 'Pulsador Industrial', precio: 12, categoria: 'Electricidad', stock: 0 },
  { id: 'P3', nombre: 'PLC Compacto', precio: 320, categoria: 'Electrónica', stock: 4 },
  { id: 'P4', nombre: 'Contactor 220V', precio: 85, categoria: 'Electricidad', stock: 15 },
  { id: 'P5', nombre: 'Relé Térmico', precio: 55, categoria: 'Electricidad', stock: 0 }
];

// 1. Filtrado de productos críticos
const criticos = inventario.filter(p => p.categoria === 'Electricidad' && p.stock === 0);

console.log('=== PRODUCTOS CRÍTICOS (Electricidad sin stock) ===');
console.table(criticos);

// 2. Notificaciones con .forEach() (Side-effect)
console.log("\n=== ALERTAS DE REPOSICIÓN ===");
```

```
criticos.forEach(producto => {  
  console.log(`[ALERTA REPOSICIÓN] El producto '${producto.nombre}' no tiene stock disponible.`);  
});
```

Ejercicio 3: Pipeline Contable de Métricas de Ventas (ejercicio03.js)

- **Objetivo:** Aplicar el encadenamiento de métodos (*Method Chaining*) procesando datos con `.filter()`, `.map()` y `.reduce()`.
- **Consigna:** Dada la lista de facturas registradas en un sistema:

JavaScript

```
const facturas = [  
  { nro: 1001, cliente: 'TechCorp', monto: 5000, pagada: true },  
  { nro: 1002, cliente: 'GlobalSvc', monto: 1200, pagada: false },  
  { nro: 1003, cliente: 'InduData', monto: 8500, pagada: true },  
  { nro: 1004, cliente: 'SistemasSA', monto: 3100, pagada: false },  
  { nro: 1005, cliente: 'PymeLab', monto: 2400, pagada: true }  
];
```

1. Construí un **Pipeline Funcional encadenado** en una sola sentencia que realice lo siguiente:
 - **Filtrar** solo las facturas que estén cobradas (`pagada === true`).
 - **Mapear** para calcular el monto final reteniendo un 21% de IVA (`monto * 1.21`).
 - **Reducir** para obtener el **monto total facturado final** acumulado.
2. Imprimí el resultado con el formato: Total cobrado con IVA: \$X.
3. Imprimí el resultado con el total con y sin IVA de las facturas pendientes de cobro.
4. Respondé en un comentario al final del código: *¿Qué ocurre si el arreglo de entrada está vacío? ¿Cómo se evita que `.reduce()` arroje un error en tiempo de ejecución?*

Solución: ejercicio03.js

```
const facturas = [  
  { nro: 1001, cliente: 'TechCorp', monto: 5000, pagada: true },  
  { nro: 1002, cliente: 'GlobalSvc', monto: 1200, pagada: false },  
  { nro: 1003, cliente: 'InduData', monto: 8500, pagada: true },  
  { nro: 1004, cliente: 'SistemasSA', monto: 3100, pagada: false },  
  { nro: 1005, cliente: 'PymeLab', monto: 2400, pagada: true }  
];
```

```
// 1. Pipeline Funcional Encadenado  
const totalCobradoConIVA = facturas
```

```
  .filter(f => f.pagada)  
  .map(f => f.monto * 1.21)  
  .reduce((acum, montoConIVA) => acum + montoConIVA, 0);
```

```
// 2. Salida
```

```
console.log(`Total cobrado con IVA: $$${totalCobradoConIVA.toFixed(2)}`);
```

```
/*
```

3. Respuesta a la pregunta conceptual:

- Si el arreglo de entrada está vacío y NO se proporciona un valor inicial a `.reduce()`, JavaScript arroja un error en tiempo de ejecución: "TypeError: Reduce of empty array with no initial value".

- Se evita pasando siempre el valor inicial explícito como segundo argumento de `.reduce()` (en este caso, 0).

```
*/
```

Ejercicio 4: Debugging de Scope, Coerción y Mutabilidad (ejercicio04.js)

- **Objetivo:** Desarrollar capacidad crítica de lectura de código ("pensar como el motor de JS") identificando errores conceptuales de tipado, *hoisting* y mutación involuntaria.
- **Consigna:** Analizá el siguiente bloque de código **sin ejecutarlo previamente**. Identificá los **3 errores lógicos/conceptuales**, explicá por qué ocurren y escribí la versión corregida:

JavaScript

// Código con errores a analizar:

// Error 1: Coerción y Scope

```
if (true) {  
  var descuento = "10";  
}
```

```
let precioFinal = 100 - descuento;
```

```
console.log("Precio con descuento:", precioFinal); // ¿Qué imprime y por qué?
```

// Error 2: Mal uso de metodos de arreglo

```
const valores = [10, 20, 30];  
const dobles = valores.map(v => {  
  v * 2;  
});
```

```
console.log("Dobles:", dobles);
```

// Error 3: Mutación no deseada

```
const configuracionInicial = { puerto: 8080, estado: 'OK' };  
const configuracionActual = configuracionInicial;  
configuracionActual.puerto = 9090;
```

```
console.log("¿El puerto inicial cambió?", configuracionInicial.puerto);
```

Ejercicio 4: ejercicio04.js (Debugging)

- **Diagnóstico de errores:**
 1. **Error 1 (Coerción y Scope):** `var descuento` se fuga fuera del `if`. En la resta `100 - "10"`, el operador `-` fuerza una coerción implícita convirtiendo `"10"` a número, dando 90. Aunque da un resultado numérico, depender de la coerción de strings y el uso de `var` es una mala práctica. Debe usarse `const/let` y tipos numéricos explícitos.

2. **Error 2 (Mal uso de .map()):** La arrow function tiene llaves { v * 2; } pero le falta la palabra clave return. Como consecuencia, .map() retorna undefined para cada elemento, resultando en [undefined, undefined, undefined].
3. **Error 3 (Mutación no deseada):** Al hacer configuracionActual = configuracionInicial, no se copia el objeto sino su **referencia en memoria**. Modificar configuracionActual.puerto muta la estructura compartida y altera configuracionInicial.puerto.

- **Versión Corregida:**

JavaScript

```
// Corregido 1: Scope adecuado y tipo numérico
if (true) {
  const descuento = 10;
  const precioFinal = 100 - descuento;
  console.log("Precio con descuento:", precioFinal); // Imprime: 90
}

// Corregido 2: Retorno explícito o sintaxis de una sola línea
const valores = [10, 20, 30];
const dobles = valores.map(v => v * 2);
console.log("Dobles:", dobles); // Imprime: [20, 40, 60]

// Corregido 3: Clonación inmutable con Spread Operator (...)
const configuracionInicial = { puerto: 8080, estado: 'OK' };
const configuracionActual = { ...configuracionInicial, puerto: 9090 };

console.log("Puerto inicial intacto:", configuracionInicial.puerto); // Imprime: 8080
console.log("Puerto nuevo actualizado:", configuracionActual.puerto); // Imprime: 9090
```

ORIGEN DE DATOS PERSISTENTE (JSON)

Una variante interesante es tomar los datos desde una fuente persistente, como un archivo JSON. Tiene como ventaja que simula un flujo real donde los datos no están *hardcodeados* en el código, sino que provienen de una fuente externa (un archivo de configuración, una base de datos o un servicio REST).

Aquí tenemos la solución completa dividida en los dos archivos necesarios y la explicación del proceso.

1. Archivo de datos: datos.json

Debe crearse en el mismo directorio donde se ejecute el script:

JSON

```
[
  { "id": 1, "nombre": "Lucas", "parcial1": 4, "parcial2": 6 },
  { "id": 2, "nombre": "Mariana", "parcial1": 8, "parcial2": 9 },
  { "id": 3, "nombre": "Gonzalo", "parcial1": 2, "parcial2": 3 },
  { "id": 4, "nombre": "Flores", "parcial1": 7, "parcial2": 5 }
]
```

2. Archivo del script: ejercicio01_json.js

En Node.js existen dos formas típicas de importar un JSON: mediante el módulo nativo del sistema de archivos (fs) o usando require(). Utilizar fs.readFileSync e JSON.parse es el estándar recomendado para enseñar procesamiento de archivos de entrada/salida.

JavaScript

```
const fs = require('fs');
const path = require('path');

// 1. Lectura sincrónica del archivo JSON desde el disco
const rutaArchivo = path.join(__dirname, 'datos.json');
const contenidoRaw = fs.readFileSync(rutaArchivo, 'utf-8');

// 2. Parseo del string JSON a un objeto/arreglo nativo de JavaScript
const estudiantes = JSON.parse(contenidoRaw);

// 3. Transformación inmutable con .map() y Spread Operator (...)
const estudiantesConPromedio = estudiantes.map(estudiante => {
  const promedio = (estudiante.parcial1 + estudiante.parcial2) / 2;
  return {
    ...estudiante,
    promedio: promedio,
    condicion: promedio >= 6 ? 'APROBADO' : 'RECUPERA'
  };
});

console.log('=== ESTUDIANTES PROCESADOS DESDE JSON ===');
console.table(estudiantesConPromedio);

// 4. Generación del marcado HTML con .map() y .join()
const itemsHTML = estudiantesConPromedio.map(
  e => `<li>${e.nombre} - Promedio: ${e.promedio} (${e.condicion})</li>`
);

const reporteHTML = `<ul class="lista-estudiantes">\n${itemsHTML.join("\n")}\n</ul>`;

console.log("\n=== REPORTE HTML GENERADO ===");
console.log(reporteHTML);

// 5. Opcional: Escribir la salida resultante en un archivo HTML en disco
fs.writeFileSync(path.join(__dirname, 'reporte.html'), reporteHTML, 'utf-8');
console.log("\n✅ Reporte guardado con éxito en \"reporte.html\"");
```

Puntos clave:

- **path.join(__dirname, 'datos.json')**: Garantiza la resolución correcta de la ruta sin importar desde qué directorio de la terminal se invoque node ejercicio01_json.js.
- **JSON.parse()**: Convierte el texto leído (string) en estructuras manipulables (arreglos y objetos de JS).
- **Inmutabilidad**: El archivo fuente original no es alterado en ningún momento.
- **fs.writeFileSync()**: Cierra el ciclo permitiendo que los alumnos vean cómo tomar datos en un formato (JSON) y exportar el resultado procesado en otro formato (HTML).